

영상 및 그래픽스 — 중간고사 파이프라인 압축 정리

1. 파이프라인 전체 흐름

Vertex Data



【1】Vertex Shader ← 정점 변환 (3D → 클립 공간)



【2】Primitive Setup ← 정점 묶어서 삼각형 만들기, Edge 생성



【3】Rasterization ← 삼각형 → Fragment, 보간



【4】Fragment Shader ← 각 Fragment 색상 계산



【5】Blending ← z-buffer 깊이 판정, Framebuffer 기록



화면 출력

🔗 파이프라인 순서는 무조건 암기. **Vertex Shader → Primitive Setup → Rasterization → Fragment Shader → Blending** 시험에서 순서 맞추기, 각 단계 역할 묻기 둘 다 나올 수 있음.

2. Vertex Shader

정점(Vertex) 하나하나를 받아서 **3D 월드 공간 → 클립 공간**으로 변환하는 단계.

입력으로 위치(Position), 색상(Color), 법선(Normal), 텍스처 좌표 등을 받고, `gl_Position`에 변환된 위치를 써서 다음 단계로 넘김.

```
layout (location = 0) in vec3 vPosition;
layout (location = 1) in vec3 vNormal;

uniform mat4 m_matrix;    // Model: 물체 자체 변환
uniform mat4 v_matrix;    // View: 카메라 변환
uniform mat4 proj_matrix; // Projection: 투영

void main() {
    gl_Position = proj_matrix * v_matrix * m_matrix * vec4(vPosition, 1.0);
}
```

🔗 행렬 곱은 **오른쪽부터** 적용된다. 코드에서 왼쪽에 써도 실제 적용 순서는 **Model → View → Projection**. `vec4(vPosition, 1.0)`에서 `w=1.0` → **Point**로 처리 (이동 적용됨). 법선벡터는 `vec4(normal, 0.0)` → **Vector**로 처리 (이동 무시됨).

🔗 `m_matrix`, `v_matrix`, `proj_matrix`를 따로 넘기는 이유: CPU 쪽에서 각각 독립적으로 바꿀 수 있어야 하기 때문. 물체는 움직이는데 카메라는 고정이라면 `m_matrix`만 업데이트하면 됨.

3. Primitive Setup

변환된 정점들을 삼각형으로 묶는 단계. Edge를 생성하고, View Frustum 밖의 삼각형을 **Clipping**함.

- 완전히 밖에 있는 삼각형 → 통째로 제거
- 부분적으로 걸쳐 있는 삼각형 → 잘라서 작은 삼각형 여러 개로 테셀레이션

🔥 Clipping을 여기서 하는 이유: 보이지도 않는 삼각형을 래스터화하면 시간 낭비. "Don't waste time rasterizing primitives you can't see!"

4. Rasterization

삼각형을 **Fragment**(픽셀 후보) 로 쪼개는 단계. 각 Fragment에 정점 데이터를 보간해서 값을 채움.

4-1. 알고리즘 흐름

```
For each Triangle:
  For each Pixel in Bounding Box:
    If Pixel in Triangle:
      Fragment 생성 + 보간
```

Bounding Box 계산 — 모든 픽셀 검사 대신 삼각형을 감싸는 최소 사각형만 검사.

```
xmin = min(ax, bx, cx),  xmax = max(ax, bx, cx)
ymin = min(ay, by, cy),  ymax = max(ay, by, cy)
```

픽셀은 **중심점(half-integer 좌표)** 기준으로 검사. 예: 픽셀 (1, 4) → 중심점 (1.5, 4.5)

4-2. Point-in-Triangle Test

외적(Cross Product)을 이용해 점 q가 삼각형 abc 안에 있는지 검사. 각 변에 대해 q가 같은 쪽에 있으면 내부.

```
(ac□ × ab□) · (ac□ × aq□) > 0  &&
(cb□ × ca□) · (cb□ × cq□) > 0  &&
(ba□ × bc□) · (ba□ × bq□) > 0
```

세 값이 모두 양수 OR 모두 음수면 내부.

🔥 b와 c 순서를 바꾸면 부호가 반전됨. 그래서 **방향(Orientation)**이 중요하고, "Just be consistent!" — 방향을 뒤로 잡든 세 값이 일관되게 같은 부호여야 함.

🔥 GPU는 Bounding Box 안의 모든 픽셀을 **병렬로 동시에** 테스트함. ac, cb, ba를 픽셀마다 공통이라 미리 계산해서 공유하고, q 부분만 병렬 처리. 현대 GPU에는 이 테스트를 위한 전용 하드웨어도 있음.

4-3. Interpolation — Barycentric Coordinates (무게중심좌표)

픽셀이 삼각형 안 어디에 위치하는지를 비율 (b1, b2, b3)로 표현하고, 이 비율로 색상·텍스처 좌표·깊이를 보간.

점의 위치: $b_1 \cdot v_1 + b_2 \cdot v_2 + b_3 \cdot v_3$

조건: $b_1 + b_2 + b_3 = 1$, 각 b 는 0~1 사이

꼭짓점 v_1 에 가까울수록 b_1 이 1에 가깝고, 정중앙은 $b_1=b_2=b_3=1/3$.

💡 $b_1+b_2+b_3 = 1$ 은 암기. 이 조건을 만족하고 모든 b 가 0 이상일 때 삼각형 내부임. 색상뿐 아니라 **깊이(z값)**도 이 방법으로 보간함 → z-buffer에 쓸 값 결정.

5. Fragment Shader

각 Fragment의 최종 색상을 계산하는 단계. 텍스처 샘플링, 조명 계산 등이 여기서 이뤄짐.

```
in vec3 Color;           // Vertex Shader에서 보간되어 넘어온 값
out vec4 FragColor;

void main() {
    FragColor = vec4(Color, 1.0);
}
```

💡 Fragment는 "Pixels-to-be" — 아직 픽셀이 아님. Blending 단계를 거쳐야 진짜 픽셀이 됨.

6. Blending

여러 Fragment 중 어느 것이 실제 화면에 그려질지 결정하는 단계.

- **Depth Test (z-buffer):** 같은 픽셀 위치에 Fragment가 여럿이면 z값이 가장 작은(앞에 있는) 것만 통과
- **Alpha Blending:** 투명도 처리 (반투명 물체)
- 통과한 Fragment를 **Framebuffer**에 기록

💡 z-buffer는 각 픽셀마다 현재 가장 앞에 있는 Fragment의 깊이를 저장하는 버퍼. 새 Fragment가 들어오면 비교해서 더 앞에 있으면 교체, 뒤에 있으면 버림.

7. Viewing — 카메라 배치

좌표계 변환 흐름

```
World Coordinates
  → (View Matrix = lookAt)
Camera Coordinates   [카메라가 원점, -z 방향을 바라봄]
  → (Projection Matrix)
Clip Coordinates     [-1,-1,-1] ~ [1,1,1] 정육면체로 압축
  → (Normalize w로 나눔)
Normalized Coordinates [-1,-1] ~ [1,1]
  → (Screen Transform)
Image Coordinates    [0,0] ~ [W,H]
```

🔗 OpenGL 기본 카메라는 원점에서 -z 방향을 바라봄. 카메라를 움직이는 게 아니라 월드 전체를 반대로 변환해서 카메라가 원점에 있는 것처럼 만드는 거임.

카메라 기저벡터 (u, v, n)

```
n = normalize(eye - center)    // 카메라 뒤쪽 방향 (-n이 바라보는 방향)
u = normalize(up × n)          // 카메라 오른쪽
v = normalize(n × u)           // 카메라 위쪽
```

View Matrix

```
| ux  uy  uz  -u·eye |
| vx  vy  vz  -v·eye |
| nx  ny  nz  -n·eye |
| 0   0   0   1     |
```

앞 3×3 = 회전 (u,v,n으로 기저 변환), 4번째 열 = 이동 (-u·eye 등)

```
glm::mat4 view = glm::lookAt(eye, center, up);
```

🔗 glm::lookAt 내부 동작 순서: ① n = eye-center 정규화 ② u = up×n 정규화 ③ v = n×u 정규화 ④ 위 행렬 조합해서 리턴. 직접 계산할 일은 없지만 시험에서 n, u, v 구하는 공식은 나올 수 있음.

8. Projection — 투영

Orthographic (직교 투영)

원근감 없음. 거리에 상관없이 크기 동일. z좌표를 버리고 x,y만 사용.

```
xp = x,   yp = y,   zp = 0
```

```
glm::mat4 proj = glm::ortho(left, right, bottom, top, zNear, zFar);
```

Perspective (원근 투영)

멀수록 작아지는 원근감. 뷰 볼륨이 절두체(Frustum) 모양.

```
// 방법 1: 직접 near plane의 범위 지정
glm::mat4 proj = glm::frustum(left, right, bottom, top, zNear, zFar);
```

```
// 방법 2: 시야각으로 지정 (더 직관적, 권장)
glm::mat4 proj = glm::perspective(fovy, aspect, zNear, zFar);
// fovy      : 수직 시야각 (라디안)
// aspect    : 화면 가로/세로 비율 = w/h
```

🔗 z-near와 z-far가 필요한 이유

- z-near 없으면: 원점 근처에서 깊이 계산 정밀도 붕괴 (0으로 나누는 문제)
- z-far 없으면: 무한히 먼 곳까지 렌더링 → 성능 문제

🔗 frustum은 near plane의 left/right/bottom/top을 직접 지정해야 해서 불편함. perspective는 시야각 (fovy)과 종횡비(aspect)만 주면 자동 계산해줘서 훨씬 편함. 실무에서는 perspective가 표준.

View Volume Normalization

Ortho든 Perspective든 투영 후 모든 가시 영역을 $[-1, -1, -1] \sim [1, 1, 1]$ 정육면체로 정규화. 그래야 이후 처리가 통일 됨.

Screen Transform

정규화 좌표 $[-1, 1] \times [-1, 1] \rightarrow$ 실제 픽셀 좌표 $[0, W] \times [0, H]$ 변환. 3단계:

1. x축 기준 반사 (상하 뒤집힘 — "Image flipped upside down")
2. (1,1)만큼 이동
3. (W/2, H/2)만큼 스케일

🔗 이미지가 뒤집히는 이유: 정규화 좌표는 y가 위로 증가하지만, 화면 픽셀 좌표는 y가 아래로 증가하기 때문.

9. 데이터 전달 — Uniform vs Attribute

구분	특징	용도
Vertex Attribute	정점마다 다른 값	위치, 색상, 법선, 텍스처 좌표
Uniform	드로우콜 전체 동일한 값	변환 행렬, 타이머, 색상 상수

Uniform 전달 코드

```
// 행렬 전달
GLint loc = glGetUniformLocation(prog, "RotationMatrix");
glUniformMatrix4fv(loc, 1, GL_FALSE, &rotationMatrix[0][0]);

// 스칼라/벡터 전달
glUniform1f(loc, val);           // float 1개
glUniform3f(loc, x, y, z);       // float 3개
glUniform4fv(loc, count, ptr);   // vec4 배열
```

🔗 glUniformMatrix4fv 의 3번째 파라미터 **Transpose**:

- GL_FALSE → Column-major order 그대로 (GLM 기본값, 보통 이걸 씀)
- GL_TRUE → Row-major order → OpenGL이 전치해서 읽음

GLM은 column-major이므로 항상 **GL_FALSE**.

🔗 `prog.setUniform("이름", 값)` 은 강의 코드에서 만든 래퍼 함수. 내부에서 `glGetUniformLocation + glUniform*` 를 순서대로 호출해주는 것.

10. VBO / VAO

개념	역할
VBO (Vertex Buffer Object)	정점 데이터를 GPU 메모리에 저장하는 버퍼
VAO (Vertex Array Object)	"이 버퍼의 데이터를 어떻게 읽어라" 는 레이아웃 정보 저장

```
// VBO: 데이터를 GPU에 올리기
glGenBuffers(1, &vbo);
glBindBuffer(GL_ARRAY_BUFFER, vbo);
glBufferData(GL_ARRAY_BUFFER, sizeof(data), data, GL_STATIC_DRAW);

// VAO: 레이아웃 정의 (location=0, vec3 타입, stride 없음)
glGenVertexArrays(1, &vao);
glBindVertexArray(vao);
glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 0, 0);
glEnableVertexAttribArray(0);
```

🔗 VAO는 "데이터 어디 있고 어떻게 읽어라"는 **설정서**. VBO는 실제 **데이터 참고**. 드로우콜 시 VAO만 바인딩하면 연결된 VBO 설정이 자동으로 따라옴.

11. GLM 주요 함수 한눈에

```
// === 변환 행렬 ===
glm::translate(m, vec3(tx, ty, tz)) // 이동
glm::rotate(m, angle_rad, vec3(ax, ay, az)) // 회전
glm::scale(m, vec3(sx, sy, sz)) // 스케일

// TRS 합성 순서 (코드 순서는 T→R→S, 실제 적용은 S→R→T)
mat4 M = glm::translate(glm::mat4(1.0f), T);
M = glm::rotate(M, angle, axis);
M = glm::scale(M, S);

// === 카메라 / 투영 ===
glm::lookAt(eye, center, up) // View Matrix
glm::perspective(fovy, aspect, near, far) // Perspective Projection
glm::ortho(l, r, b, t, near, far) // Orthographic Projection
glm::frustum(l, r, b, t, near, far) // Perspective (직접 지정)

// === 기타 ===
glm::mat4(1.0f) // 단위행렬
glm::radians(45.0f) // 도 → 라디안 변환
glm::transpose(m) // 전치
```

🔗 `glm::rotate` 의 angle은 **라디안** 단위. 도(degree)로 쓰면 `glm::radians()` 로 변환 필수.

12. 시험 직전 핵심 암기

🔥 **파이프라인 5단계 순서** Vertex Shader → Primitive Setup → Rasterization → Fragment Shader → Blending

🔥 **Vertex Shader 행렬 곱 순서** `gl_Position = proj * view * model * vec4(pos, 1.0)` 오른쪽부터 적용: Model → View → Projection

🔥 **w값 구분**

- $w = 1 \rightarrow$ Point (이동 O)
- $w = 0 \rightarrow$ Vector (이동 X, 법선벡터 등)

🔥 **Barycentric 조건** $b_1 + b_2 + b_3 = 1$, 각 $b \in [0,1]$ 이면 삼각형 내부

🔥 **lookAt 기저벡터 공식** $n = \text{normalize}(\text{eye} - \text{center})$ $u = \text{normalize}(up \times n)$ $v = \text{normalize}(n \times u)$

🔥 **glUniformMatrix4fv Transpose 파라미터** GLM 사용 시 항상 `GL_FALSE` (column-major)

🔥 **Clipping 목적** Frustum 밖 삼각형 제거 → 보이지 않는 것 래스터화 안 함 → 성능 절약

🔥 **Point-in-Triangle 판정** 외적 후 내적인 세 값이 모두 같은 부호(모두 양수 or 모두 음수)면 내부 방향 (Orientation)이 바뀌어도 되지만 **일관성** 유지 필수